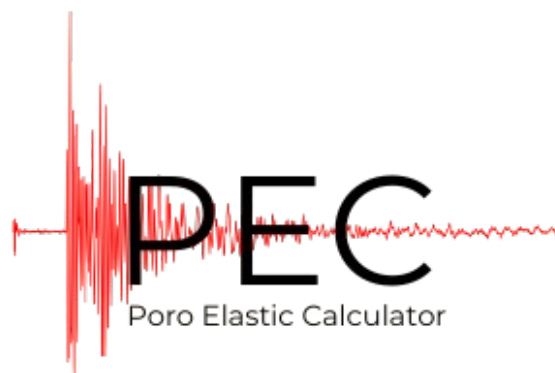




Manual Técnico

V 3.0

Set/2025

Sumário

1	Visão Geral e Arquitetura	2
1.1	Estrutura de Classes	2
2	Motor Físico: Teoria do Meio Efetivo	2
2.1	Modelo Matemático	3
2.2	Tratamento da Porosidade	3
3	Implementação e Algoritmos	3
3.1	Fluxo de Execução	3
3.2	Snippet: Polimorfismo no Cálculo	4
4	Sistema de Visualização	4
4.1	Automação com Gnuplot	5
4.2	Gestão de Arquivos de Saída	5

1 Visão Geral e Arquitetura

O **PoreElasticCalculator (PEC)** é uma aplicação de engenharia desenvolvida em C++17 para modelagem de física de rochas. Sua arquitetura prioriza a separação entre a interface de usuário, a lógica matemática e a camada de visualização.

1.1 Estrutura de Classes

O sistema adota padrões de projeto modernos para garantir extensibilidade:

- **Facade (CSimulador):** Centraliza o controle, simplificando o `main.cpp`.
- **Strategy (CElasticCalculator):** Permite a troca dinâmica dos algoritmos de cálculo (Bulk ou Shear) sem alterar o código cliente.
- **Singleton/Repository (CMineralDatabase):** Garante um ponto único de acesso às propriedades dos minerais carregadas do arquivo CSV.

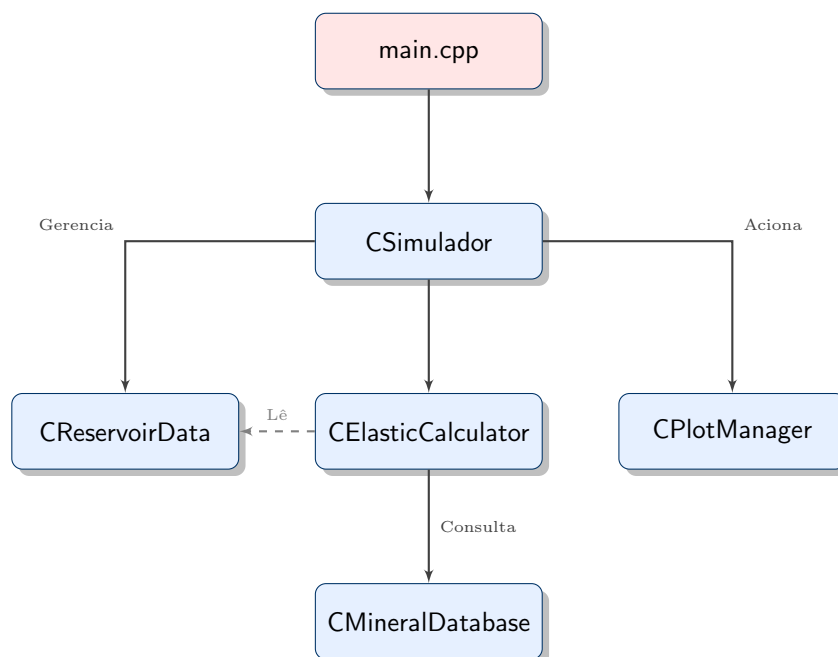


Figura 1: Arquitetura de alto nível e fluxo de dependências.

2 Motor Físico: Teoria do Meio Efetivo

O núcleo matemático do PEC implementa as médias de Voigt-Reuss-Hill para meios porosos isotrópicos.

2.1 Modelo Matemático

A rocha é tratada como um compósito de N minerais e uma fase de poro (ϕ).

1. **Voigt (Iso-deformação):** Limite superior de rigidez.

$$M_{Voigt} = \sum f_i M_i \quad (\text{Onde } M = K \text{ ou } G) \quad (1)$$

2. **Reuss (Iso-tensão):** Limite inferior de rigidez.

$$\frac{1}{M_{Reuss}} = \sum \frac{f_i}{M_i} \quad (2)$$

3. **Hill:** Média aritmética usada como estimativa final.

$$M_{Hill} = \frac{M_{Voigt} + M_{Reuss}}{2} \quad (3)$$

2.2 Tratamento da Porosidade

A porosidade é integrada ao cálculo como uma fase mineral com propriedades elásticas nulas ($K_{poro} \approx 0, G_{poro} \approx 0$). O algoritmo normaliza as frações da matriz sólida ($1 - \phi$) antes de combinar com o poro, garantindo consistência física.

3 Implementação e Algoritmos

3.1 Fluxo de Execução

O diagrama abaixo ilustra o ciclo de vida de uma solicitação de cálculo, desde a entrada do usuário até a saída dos dados.

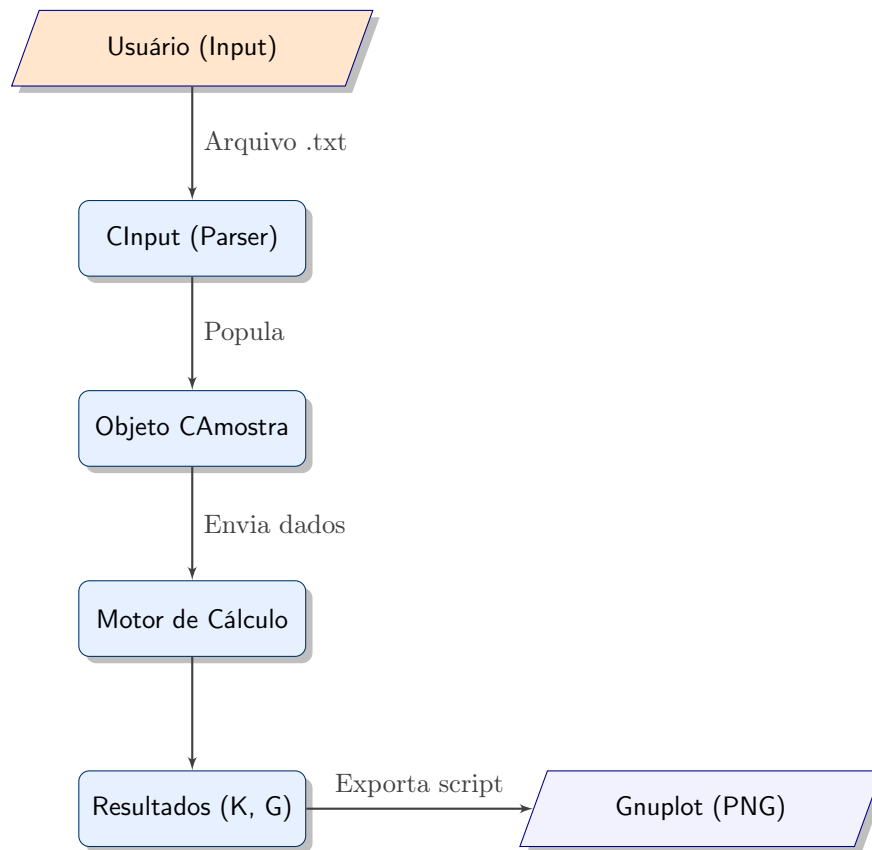


Figura 2: Pipeline de processamento de dados.

3.2 Snippet: Polimorfismo no Cálculo

A classe base `CElasticCalculator` define a interface virtual pura que obriga a implementação do método `Calcular` nas classes derivadas.

```

1 class C ElasticCalculator {
2 public:
3     virtual ~CElasticCalculator() = default;
4     // Metodo virtual puro: obriga implementacao nas derivadas
5     virtual double Calcular(const CAmostra& amostra,
6                             const CMineralDatabase& db) const = 0;
7 };
  
```

Listing 1: Interface para o Strategy Pattern.

4 Sistema de Visualização

A geração de gráficos é gerenciada pela classe `CPlotManager`, que abstrai a complexidade do Gnuplot.

4.1 Automação com Gnuplot

O sistema não requer interação manual com o Gnuplot. O processo é totalmente automatizado:

1. **Geração de Dados:** O C++ exporta arquivos `.dat` temporários na pasta `test/Plots/`.
2. **Scripting:** Um arquivo `.gp` é criado dinamicamente com as configurações de estilo (eixos, grid, títulos).
3. **Renderização:** O comando `system("gnuplot script.gp")` é invocado para gerar a imagem PNG.

4.2 Gestão de Arquivos de Saída

Para evitar sobrescrita de dados em execuções consecutivas, o sistema utiliza *timestamps* (carimbos de tempo) nos nomes dos arquivos.

```
1 std::string timestamp = getCurrentTimestamp(); // Retorna  
   YYYYMMDD_HHMMSS  
2 std::string filename = "Grafico_K_vs_Porosidade_" + timestamp + ".png";
```

Listing 2: Geração de nomes únicos para arquivos.